Cryptographic Storage Cheat Sheet¶

Introduction¶

This article provides a simple model to follow when implementing solutions to protect data at rest.

Passwords should not be stored using reversible encryption - secure password hashing algorithms should be used instead. The Password Storage Cheat Sheet contains further guidance on storing passwords.

Architectural Design¶

The first step in designing any application is to consider the overall architecture of the system, as this will have a huge impact on the technical implementation.

This process should begin with considering the threat model of the application (i.e, who you are trying to protect that data against).

The use of dedicated secret or key management systems can provide an additional layer of security protection, as well as making the management of secrets significantly easier - however it comes at the cost of additional complexity and administrative overhead - so may not be feasible for all applications. Note that many cloud environments provide these services, so these should be taken advantage of where possible. The Secrets Management Cheat Sheet contains further guidance on this topic.

Where to Perform Encryption¶

Encryption can be performed on a number of levels in the application stack, such as:

- At the application level.
- At the database level (e.g, SQL Server TDE)
- At the filesystem level (e.g, BitLocker or LUKS)
- At the hardware level (e.g, encrypted RAID cards or SSDs)

Which layer(s) are most appropriate will depend on the threat model. For example, hardware level encryption is effective at protecting against the physical theft of the server, but will provide no protection if an attacker is able to compromise the server remotely.

Minimize the Storage of Sensitive Information¶

The best way to protect sensitive information is to not store it in the first place. Although this applies to all kinds of information, it is most often applicable to credit card details, as they are highly desirable for attackers, and PCI DSS has such stringent requirements for how they must be stored. Wherever possible, the storage of sensitive information should be avoided.

Algorithms¶

For symmetric encryption **AES** with a key that's at least **128 bits** (ideally **256 bits**) and a secure mode should be used as the preferred algorithm.

For asymmetric encryption, use elliptical curve cryptography (ECC) with a secure curve such as **Curve25519** as a preferred algorithm. If ECC is not available and **RSA** must be used, then ensure that the key is at least **2048 bits**.

Many other symmetric and asymmetric algorithms are available which have their own pros and cons, and they may be better or worse than AES or Curve25519 in specific use cases. When considering these, a number of factors should be taken into account, including:

- Key size.
- Known attacks and weaknesses of the algorithm.
- Maturity of the algorithm.
- Approval by third parties such as NIST's algorithmic validation program.
- Performance (both for encryption and decryption).
- Quality of the libraries available.
- Portability of the algorithm (i.e, how widely supported is it).

In some cases there may be regulatory requirements that limit the algorithms that can be used, such as FIPS 140-2 or PCI DSS.

Custom Algorithms¶

Don't do this.

Cipher Modes¶

There are various modes that can be used to allow block ciphers (such as AES) to encrypt arbitrary amounts of data, in the same way that a stream cipher would. These modes have different security and performance characteristics, and a full discussion

of them is outside the scope of this cheat sheet. Some of the modes have requirements to generate secure initialization vectors (IVs) and other attributes, but these should be handled automatically by the library.

Where available, authenticated modes should always be used. These provide guarantees of the integrity and authenticity of the data, as well as confidentiality. The most commonly used authenticated modes are **GCM** and **CCM**, which should be used as a first preference.

If GCM or CCM are not available, then CTR mode or CBC mode should be used. As these do not provide any guarantees about the authenticity of the data, separate authentication should be implemented, such as using the Encrypt-then-MAC technique. Care needs to be taken when using this method with variable length messages

ECB should not be used outside of very specific circumstances.

Random Padding¶

For RSA, it is essential to enable Random Padding. Random Padding is also known as OAEP or Optimal Asymmetric Encryption Padding. This class of defense protects against Known Plain Text Attacks by adding randomness at the beginning of the payload.

The Padding Schema of PKCS#1 is typically used in this case.

Secure Random Number Generation¶

Random numbers (or strings) are needed for various security critical functionality, such as generating encryption keys, IVs, session IDs, CSRF tokens or password reset tokens. As such, it is important that these are generated securely, and that it is not possible for an attacker to guess and predict them.

It is generally not possible for computers to generate truly random numbers (without special hardware), so most systems and languages provide two different types of randomness.

Pseudo-Random Number Generators (PRNG) provide low-quality randomness that are much faster, and can be used for non-security related functionality (such as ordering results on a page, or randomizing UI elements). However, they **must not** be used for anything security critical, as it is often possible for attackers to guess or predict the output.

Cryptographically Secure Pseudo-Random Number Generators (CSPRNG) are designed to produce a much higher quality of randomness (more strictly, a greater amount of entropy), making them safe to use for security-sensitive functionality. However, they are slower and more CPU intensive, can end up blocking in some circumstances when large amounts of random data are requested. As such, if large amounts of non-security related randomness are needed, they may not be appropriate.

The table below shows the recommended algorithms for each language, as well as insecure functions that should not be used.

Language	**Unsafe Functions**	**Cryptographically Secure Functions**
C	random(), rand()	getrandom(2)
Java	Math.random(), StrictMath.random(), java.util.Random, java.util.SplittableRandom, java.util.concurrent.ThreadLocalRandom	java.security.SecureRandom, java.util.UUID.randomUUID()
PHP	array_rand(), lcg_value(), mt_rand(), rand(), uniqid()	Random\Engine\Secure in PHP 8.2, random_bytes(), random_int() in PHP 7, openssl_random_pseudo_bytes() in PHP 5
.NET/C#	Random()	RandomNumberGenerator
Objective-C	arc4random()/arc4random_uniform() (Uses RC4 Cipher), subclasses ofGKRandomSource, rand(), random()	SecRandomCopyBytes
Python	random()	secrets()
Ruby	rand(), Random	SecureRandom
Go	rand using math/rand package	crypto.rand package
Rust	rand::prng::XorShiftRng	rand::prng::chacha::ChaChaRng and the rest of the Rust library CSPRNGs.

Node.js	Math.random()	crypto.randomBytes(), crypto.randomInt(), crypto.randomUUID()

UUIDs and GUIDs¶

Universally unique identifiers (UUIDs or GUIDs) are sometimes used as a quick way to generate random strings. Although they can provide a reasonable source of randomness, this will depend on the type or version of the UUID that is created.

Specifically, version 1 UUIDs are comprised of a high precision timestamp and the MAC address of the system that generated them, so are **not random** (although they may be hard to guess, given the timestamp is to the nearest 100ns). Type 4 UUIDs are randomly generated, although whether this is done using a CSPRNG will depend on the implementation. Unless this is known to be secure in the specific language or framework, the randomness of UUIDs should not be relied upon.

Defense in Depth¶

Applications should be designed to still be secure even if cryptographic controls fail. Any information that is stored in an encrypted form should also be protected by additional layers of security. Application should also not rely on the security of encrypted URL parameters, and should enforce strong access control to prevent unauthorized access to information.

Key Management¶

Processes¶

Formal processes should be implemented (and tested) to cover all aspects of key management, including:

- Generating and storing new keys.
- Distributing keys to the required parties.
- Deploying keys to application servers.
- Rotating and decommissioning old keys

Key Generation¶

Keys should be randomly generated using a cryptographically secure function, such as those discussed in the Secure Random Number Generation section. Keys **should**

not be based on common words or phrases, or on "random" characters generated by mashing the keyboard.

Where multiple keys are used (such as data separate data-encrypting and key-encrypting keys), they should be fully independent from each other.

Key Lifetimes and Rotation¶

Encryption keys should be changed (or rotated) based on a number of different criteria:

- If the previous key is known (or suspected) to have been compromised.
    - This could also be caused by a someone who had access to the key leaving the organization.
- After a specified period of time has elapsed (known as the cryptoperiod).
    - There are many factors that could affect what an appropriate cryptoperiod is, including the size of the key, the sensitivity of the data, and the threat model of the system. See section 5.3 of NIST SP 800-57 for further guidance.
- After the key has been used to encrypt a specific amount of data.
    - This would typically be 2^35 bytes (~34GB) for 64-bit keys and 2^68 bytes (~295 exabytes) for 128-bit block size.
- If there is a significant change to the security provided by the algorithm (such as a new attack being announced).

Once one of these criteria have been met, a new key should be generated and used for encrypting any new data. There are two main approaches for how existing data that was encrypted with the old key(s) should be handled:

1. Decrypting it and re-encrypting it with the new key.
2. Marking each item with the ID of the key that was used to encrypt it, and storing multiple keys to allow the old data to be decrypted.

The first option should generally be preferred, as it greatly simplifies both the application code and key management processes; however, it may not always be feasible. Note that old keys should generally be stored for a certain period after they have been retired, in case old backups of copies of the data need to be decrypted.

It is important that the code and processes required to rotate a key are in place **before** they are required, so that keys can be quickly rotated in the event of a compromise. Additionally, processes should also be implemented to allow the encryption algorithm or library to be changed, in case a new vulnerability is found in the algorithm or implementation.

Key Storage¶

Securely storing cryptographic keys is one of the hardest problems to solve, as the application always needs to have some level of access to the keys in order to decrypt the data. While it may not be possible to fully protect the keys from an attacker who has fully compromised the application, a number of steps can be taken to make it harder for them to obtain the keys.

Where available, the secure storage mechanisms provided by the operating system, framework or cloud service provider should be used. These include:

- A physical Hardware Security Module (HSM).
- A virtual HSM.
- Key vaults such as Amazon KMS or Azure Key Vault.
- An external secrets management service such as Conjur or HashiCorp Vault.
- Secure storage APIs provided by the ProtectedData class in the .NET framework.

There are many advantages to using these types of secure storage over simply putting keys in configuration files. The specifics of these will vary depending on the solution used, but they include:

- Central management of keys, especially in containerised environments.
- Easy key rotation and replacement.
- Secure key generation.
- Simplifying compliance with regulatory standards such as FIPS 140 or PCI DSS.
- Making it harder for an attacker to export or steal keys.

In some cases none of these will be available, such as in a shared hosting environment, meaning that it is not possible to obtain a high degree of protection for any encryption keys. However, the following basic rules can still be followed:

- Do not hard-code keys into the application source code.
- Do not check keys into version control systems.
- Protect the configuration files containing the keys with restrictive permissions.
- Avoid storing keys in environment variables, as these can be accidentally exposed through functions such as phpinfo() or through the /proc/self/environ file.

The Secrets Management Cheat Sheet provides more details on securely storing secrets.

Separation of Keys and Data¶

Where possible, encryption keys should be stored in a separate location from encrypted data. For example, if the data is stored in a database, the keys should be stored in the filesystem. This means that if an attacker only has access to one of these (for example through directory traversal or SQL injection), they cannot access both the keys and the data.

Depending on the architecture of the environment, it may be possible to store the keys and data on separate systems, which would provide a greater degree of isolation.

Encrypting Stored Keys¶

Where possible, encryption keys should themselves be stored in an encrypted form. At least two separate keys are required for this:

- The Data Encryption Key (DEK) is used to encrypt the data.
- The Key Encryption Key (KEK) is used to encrypt the DEK.

For this to be effective, the KEK must be stored separately from the DEK. The encrypted DEK can be stored with the data, but will only be usable if an attacker is able to also obtain the KEK, which is stored on another system.

The KEK should also be at least as strong as the DEK. The envelope encryption guidance from Google contains further details on how to manage DEKs and KEKs.

In simpler application architectures (such as shared hosting environments) where the KEK and DEK cannot be stored separately, there is limited value to this approach, as an attacker is likely to be able to obtain both of the keys at the same time. However, it can provide an additional barrier to unskilled attackers.

A key derivation function (KDF) could be used to generate a KEK from user-supplied input (such a passphrase), which would then be used to encrypt a randomly generated DEK. This allows the KEK to be easily changed (when the user changes their passphrase), without needing to re-encrypt the data (as the DEK remains the same).